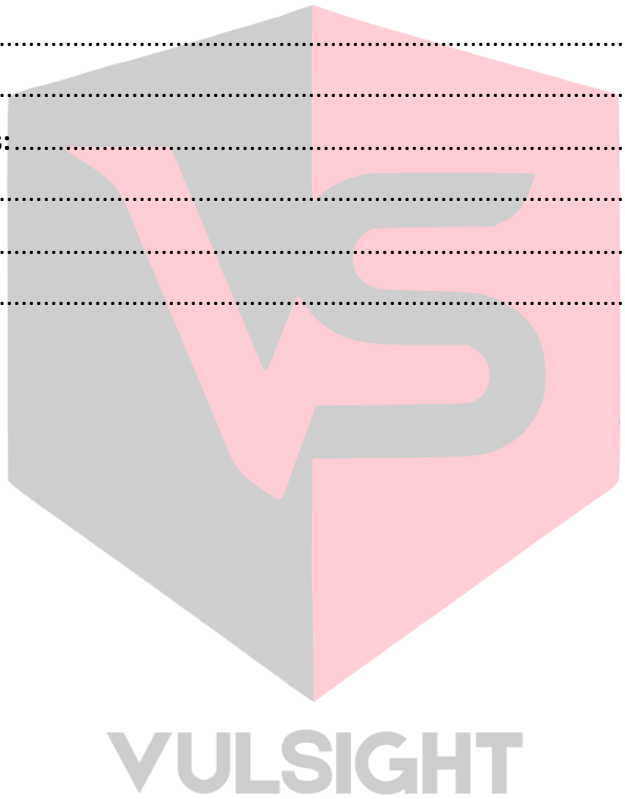
SMART CONTRACT AUDIT

Conducted for: SoulVerse

VulSight
31/5/2024

Contents



Executive Summary:

The client contacted our Consulting Firm to conduct a smart contract audit on their Solidity based smart contracts. The contract that was to be audited is a contract which powers the staking mechanism within the SoulVerse ecosystem.

Upon thorough review and analysis of the provided staking contract code, no significant vulnerabilities or security issues were identified. The contract adheres to best practices and implements appropriate safeguards to ensure the integrity and security of the staking process.

The contract allows users to stake tokens for predefined durations (3 months, 6 months, and 12 months) with corresponding annual percentage returns (APRs) of 200%, 250%, and 350%, respectively. The staking and unstaking functionalities operate as intended, with the correct calculation of staking rewards based on the amount staked and the respective APR.

Project Background:

The smart contracts are written in solidity. The token contract is a simple staking contract. The contract is deployed on the Polygon Amoy Testnet.

Audit Scope:

Deployed Addresses	**0x1efe491571bd423a448165c99131ddb0e8471e61**
Chain	**Polygon Amoy Testnet**
Language	**Solidity**
Audit Completion Date	**31/5/24**

Audit Summary:

- **Secure token transactions**: The contract utilizes the OpenZeppelin library's `SafeERC20` functions to handle token transfers securely, mitigating the risks associated with token transfers.
- **Access control:** The contract employs the `Ownable` contract from the OpenZeppelin library to restrict access to critical functions, such as modifying time frames and APRs, to the contract owner only. This ensures that only authorized individuals can make changes to the contract's parameters.

- **Correct reward calculations:** The `calculateYield` function accurately calculates the staking rewards based on the staked amount and the corresponding APR for the selected duration. Manual calculations and simulations confirm that the rewards are distributed correctly.
- **Proper input validation:** The contract includes necessary input validations, such as checking for non-zero staking amounts and valid staking durations. It also verifies that stakes have matured before allowing users to unstake their tokens.
- **Clear and auditable code:** The contract code is well-structured, properly commented, and follows Solidity conventions, making it readable and easier to audit.

Various tools such as Slither and different LLM scanners were used in the audit. Extensive manual code review was also done to ensure that any vulnerability if present could be detected in the audit.

We found:

- **0 Critical risk vulnerabilities**
- **0 High risk vulnerabilities**
- **0 Medium risk vulnerabilities**
- **0 Low risk vulnerabilities**

Severity Definitions:

Risk Level	Description
Critical	**Any kind of vulnerability that could lead to direct token or monetary loss**
High	**Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.**
Medium	**Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss**
Low	**Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract**

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Matching Project Specifications	Passed
Logical Vulnerabilities	Passed

Function visibility not explicitly declared	Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed
Insufficient Decentralization	Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included one smart contract, which are written in Solidity programming language. The code is well indented and clear in nature.

Audit Findings:

Critical vulnerabilities:

Zero critical vulnerabilities were found in the smart contract.

High vulnerabilities:

Zero high vulnerabilities were found in the smart contract.

Medium vulnerabilities:

Zero medium vulnerabilities were found in the smart contract.

Low vulnerabilities:

Zero low vulnerabilities were found in the smart contract.

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. It is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.

Conclusion:

The smart contract was tested extensively both automatically and manually. No Vulnerabilities were discovered within it. It is recommended that additional security measures may be undertaken to ensure future security such that of a creation of a bug bounty program.